ResultLens – Project Architecture (Binding Contract)

> This file is the single source of truth. Do not deviate without explicit user approval.
> Stack, structure, naming, and RBAC below are FIXED.

1. Overview

ResultLens – GGSIPU Result Analyzer & CGPA Calculator

- Ingest a single GGSIPU result PDF (one file, may contain one semester).
- Extract every subject's marks, compute SGPA per semester + CGPA across ingested semesters.
- Determine toppers: subject-wise, college-wise, semester-wise, overall (university).

2. Tech Stack (Fixed – No Substitution)

Layer	Choice
Frontend	React (Vite) + React Router + Axios + TanStack Query + react-icons + **Tailwind CSS** (single styling system, no CSS Modules mixing)
Backend	Node.js + Express
DB	PostgreSQL via `pg` Pool + thin query layer (no heavy ORM). Knex allowed only for migrations if needed, otherwise plain `node-pg-migrate` -style SQL files.
Auth	JWT in `httpOnly` cookie + bcrypt (`bcryptjs`)
PDF parsing	`pdfjs-dist` (primary) with `pdf-parse` fallback evaluation in Phase 3 – decision logged here before use
Validation	Zod on every request body/query/params
Uploads	multer memory storage, PDF-only (`application/pdf`), size-limited (10 MB default via env)
Tests	Vitest (backend + frontend utils), Supertest for API smoke in later phases
Lint/Format	ESLint + Prettier for both `client/` and `server/`

3. Monorepo Layout

```
result-analysis/
├─ ARCHITECTURE.md          # this file (binding)
├─ README.md                # setup, env, migrations, API list, diagram (Phase 8)
├─ client/                  # Vite React app
└─ server/
   ├─ src/
   │  ├─ app.js
   │  ├─ server.js
   │  ├─ config/            # env.js, db.js, constants.js
   │  ├─ routes/            # auth.routes.js, upload.routes.js, student.routes.js, topper.routes.js, an
   │  ├─ controllers/       # thin only
   │  ├─ services/          # business logic only
   │  ├─ repositories/      # ALL SQL lives here
   │  ├─ parsers/           # pdfTextExtractor.js, resultParser.js, lineMatchers.js
   │  ├─ middlewares/       # auth.js, requireRole.js, validate.js, upload.js, errorHandler.js, notFoun
   │  ├─ validators/        # zod schemas per route
   │  ├─ utils/             # see §7
   │  └─ db/migrations/ + seeds/
   ├─ tests/                # unit tests
   └─ .env.example
```

Frontend (industry-standard, feature-based):

```
client/src/
├─ app/            # App.jsx, router.jsx, providers.jsx (QueryClient, AuthProvider)
├─ assets/
├─ components/
│  ├─ ui/         # Button, Input, Select, Modal, Card, Tabs, Badge, Table, Spinner, EmptyState, Toast,
│  ├─ layout/     # Header, Navbar, Footer, PageContainer, ProtectedRoute
│  └─ feedback/   # ErrorBoundary, Loader
├─ features/
│  ├─ auth/       # FacultyLoginForm, hooks/useAuth, api/, context/ (faculty/admin only – no student au
│  ├─ upload/     # PdfDropzone, UploadProgress, api/, hooks/
│  ├─ student/    # StudentSearch, StudentSummaryCard, MarksTable, hooks/, api/
│  ├─ toppers/    # TopperTabs, Podium, TopperTable, hooks/, api/
│  └─ analytics/  # CollegeSgpaChart, SubjectPassChart, hooks/, api/
├─ pages/         # HomePage, StudentResultPage, ToppersPage, AnalyticsPage, NotFoundPage
├─ hooks/         # useDebounce, useLocalStorage, useDisclosure
├─ services/      # axiosInstance.js (withCredentials + interceptors)
├─ utils/         # formatters.js, constants.js, gradeColors.js, validators.js, cn.js
├─ routes/        # route constants + guards
├─ styles/        # tokens (CSS vars), globals, theme
└─ main.jsx
```

Rules:

- Functional components + hooks only. One component per file. PascalCase components, `useXxx` hooks.
- TanStack Query inside feature hooks only; components never call Axios directly.
- Shared `Table` used by both Toppers table and student marks table. No duplicated markup.
- Icons: `react-icons` only (`fi` , `md`). No emojis / inline SVG for icons.
- Forms: `react-hook-form` + `zod` resolver, same rules as backend.

4. RBAC + Access Contract (Binding – revised per user: NO student login)

Terminology: **Professor == Faculty**. Single role string `'faculty'` in DB/JWT. No separate `professor` role. There is **no student account, no student signup, no student login**. Students are rows in `students` (extracted from the PDF), not users. Anyone can look up a result by enrollment number + semester.

Capability	public (no login)	faculty (professor)	admin
`POST /api/v1/auth/login`	– (no student login exists)	yes (`username` (=ID) + `password` via **Faculty Login**)	yes (`username` + `password` ; role resolved from JWT)

Capability	public (no login)	faculty (professor)	admin
`POST /api/v1/admin/faculty` create professor	no	no	**ALLOWED only** (fields `username` (=ID) + `password` + optional `name` ; role forced `'faculty'`)
`POST /api/v1/uploads` (PDF)	no (401)	**ALLOWED only**	allowed (fallback)
`GET /api/v1/uploads` history	no	allowed	allowed
`GET /api/v1/students/:rollNo` result lookup	**yes** – enrollment no `+ ?semester=`	yes	yes
`GET /api/v1/students?query=` search	**yes** (public search)	yes	yes
`GET /api/v1/toppers/*`	**yes** (public, full ranked lists)	yes	yes
`GET /api/v1/analytics/*`	**yes** (public)	yes	yes
`GET /api/v1/meta/*` filters	**yes** (public)	yes	yes
Seed/create faculty accounts	no	no	yes (seed script + `POST /admin/faculty` panel, no self-signup)

Login UI: single **Faculty Login** form (`ID/username + password`). No student tabs, no signup form.
Public home page shows the result lookup (enrollment number + semester selector) for everyone.

Auth mechanics:

- JWT payload: `{ sub: userId, role, username }` , short expiry (e.g. `JWT_EXPIRES_IN=1d` via env).
- Transport: `httpOnly; SameSite=Lax; Secure` (secure in prod) cookie `access_token` . `GET /auth/me` resolves session from cookie.
- `auth.js` middleware: verify cookie → attach `req.user` . `requireRole('faculty')` / `requireRole('admin')` – no inline role checks. Public lookup/topper/analytics routes carry NO auth middleware.
- Rate-limit auth routes (`express-rate-limit`). `helmet` + CORS with `credentials:true` and explicit `CLIENT_ORIGIN` .
- Passwords: bcrypt hash (cost 10–12 via env). Never return `password_hash` .
- Frontend: `ProtectedRoute { allowedRoles: ['faculty','admin'] }` guards Upload + Admin pages only. Result, Toppers and Analytics pages are public. Faculty-only dropzone; visitors see the public lookup

form. Axios `withCredentials: true`.

5. Source PDF Format (Parser Must Handle)

PDF alternates:

1. **Scheme page** per institution: `Institution Code: 142 Institution: <NAME>`, `SchemeID`, `Sem./Year`, papers table row e.g.:
   `01 020301 BCA301 OPERATING SYSTEM & LINUX PROGRAMMING 4 THEORY UES COMPULSORY DROPPABLE 25 75 100 40`
   Fields: S.No, Paper ID, Code, Subject, Credit, Type, Exam Mode, Kind, Minor, Major, Max, Pass. Minor may be `--`.
2. **Result Tabulation pages**: `Batch: 2022 Examination: REGULAR December, 2024` (or `REAPPEAR`), student blocks:

```
00111102022PRIYAM KUMAR JHASID: 480000053814SchemeID: 480202021001
020301(4)
 21 63
84(A+)
...
 27   <- credits secured (CS)
```

Parsing rules:

- Roll = first 11 digits (`SSS III PPP BB`). Name = remainder until `SID:`. Validate via `utils/rollNumber.js`.
- Paper entry: `PAPERID(credits)` → `internal external` → `total(grade)`.
- Absent: `A` in external and `A` total/grade. Internal may be `-` (e.g. summer training).
- Grace: `40(P)*` → `is_grace=true`.
- Robust to `A(F)`, `CS = 0`; never crash – mark absent/backlog.
- Papers keyed by `(scheme_id, paper_id)` – names/credits may differ per scheme.
- Institution keyed by **code** (names may alias; same name may map to multiple codes).
- Page-by-page with carried context `{ institution, scheme, batch, exam }`. Pure function: `parse(text) → structured object`. No I/O in parser.
- Idempotent: SHA-256 dedupe (`uploads.file_hash` UNIQUE → 409 on re-upload); re-processing uses upserts.

6. Database (PostgreSQL – Exact Table Names)

Tables: `programmes`, `institutions`, `schemes`, `papers`, `grade_scale`, `students`, `uploads`, `exams`, `student_results`, `marks`, `semester_summary`, `users`.

Key constraints:

- `papers UNIQUE(scheme_id, paper_id)` + fields: code, name, credits, paper_type, exam_mode, kind, minor_max, major_max, max_marks, pass_marks.
- `marks(result_id, paper_pk, internal, external NULLABLE, total, grade, is_absent, is_grace)` + `UNIQUE(result_id, paper_pk)`.
- `student_results UNIQUE(exam_id, roll_no)`, `exam_type REGULAR/REAPPEAR`, `credits_secured`.
- `semester_summary PK(roll_no, semester)`: sgpa, cgpa, total_credits, has_backlog, college_rank, university_rank.
- `users(role IN ('faculty','admin'), username UNIQUE, password_hash)` (+ optional `name`, `email`). No student users.
- `grade_scale` seeded: O 90–100=10, A+ 75–89=9, A 65–74=8, B+ 55–64=7, B 50–54=6, C 45–49=5, P 40–44=4, F/Ab <40=0. **Assumption – kept as data, never hard-coded** (code comment required).
- Indexes: `marks(paper_pk, total DESC)`, `students(institution_code)`, `student_results(exam_id)`.
- Ingestion: batched inserts in single transaction per upload.

7. Calculation Rules (Only in `utils/calculations.js`, Pure + Unit-Tested)

- `gradePoint(total|grade)` reads `grade_scale` data (injected, not hard-coded).
- `SGPA = Σ(credits × gp) / Σ(credits)` per semester. Failed/absent = gp 0. Round 2 decimals.
- Reappear: latest attempt per paper wins.
- `CGPA = Σ(SGPA_sem × credits_sem) / Σ(credits_sem)`.
- `has_backlog = any latest-attempt grade F or absent`.
- Ranking in SQL with `RANK() OVER (PARTITION BY ... ORDER BY ...)`:
  college → `PARTITION BY institution`, university → global, semester topper → per semester, subject topper → `MAX(total)` per paper (return all ties), overall → max CGPA.
- Recompute `semester_summary` after each successful upload (transactional service).

8. Backend Conventions (Binding)

- `controllers/` thin: validate → service → `apiResponse`. No SQL, no business rules.
- `services/` business logic + transactions. `repositories/` ALL SQL, one file per domain, parameterized queries only.
- `utils/`: `asyncHandler.js`, `ApiError.js`, `apiResponse.js`, `calculations.js`, `rollNumber.js`, `hash.js`, `pagination.js`, `rankingSql.js`, `dateFormat.js`, `logger.js`. Anything used twice → helper.
- Central `errorHandler.js` + `notFound.js`. All errors as `ApiError`. Response shape `{ success, data, message }`.
- `validate.js` (Zod) on body/query/params. `upload.js` (multer memory, PDF-only, limit). Env-driven config, no secrets in code.

9. REST API (`/api/v1`)

- `POST /auth/login` (faculty/admin only: `{ username, password }` – no student login, no signup)
- `POST /auth/logout` , `GET /auth/me`
- `POST /admin/faculty` (admin only – create professor `{ username, password, name? }`)
- `POST /uploads` (faculty/admin, `multipart/form-data field=pdf`) → `{ uploadId, status }` ; `GET /uploads` (faculty/admin)
- `GET /students/:rollNo?semester=` → **public** result lookup: profile + per-sem marks + SGPA/CGPA + ranks
- `GET /students?query=&institution=&semester=&page=&limit=` → **public** search
- `GET /toppers/overall` | `/toppers/semester/:sem` | `/toppers/college/:code` | `/toppers/subject/:paperId` → **public**, full ranked lists
- `GET /analytics/summary` | `/analytics/college-sgpa` | `/analytics/subject-pass-rate`
- `GET /meta/institutions` | `/meta/papers` | `/meta/semesters`

All list endpoints: `{ success, data: { items, page, limit, total }, message }` .

10. UI Screens (Binding)

Theme: brand `ResultLens` , primary indigo `#2b3a8c / #4b5fd6` , accent amber `#f5a524` , Manrope font, soft-border cards, sticky header + **Sign in / Login** top-right.

1. **Home / Result lookup (public)**: hero, enrollment-number + semester form → result card; faculty sees the `Result PDF` dropzone instead/below; 4-step strip (Upload → Extract → Calculate → Rankings), stat cards.
2. **Student Result (public, no login)**: SGPA per semester + CGPA + college/university rank card; subject-wise marks table (Int., Ext., Total, Grade pill via shared `Table`).
3. **Toppers (public)**: tabs Overall / Semester-wise / College-wise / Subject-wise; podium top-3 + full ranked table (name, CGPA/marks) with filters (college, semester, subject).
4. **Analytics (public)**: avg SGPA by college, pass % by subject, overall pass rate, avg CGPA, highest CGPA, backlog count.
5. **Faculty Login**: single form (ID/username + password) + inline errors, focus-trapped modal, Esc close. No signup UI.
   Global: loading/empty/error states every screen; mobile-first responsive; light/dark via CSS vars; labelled inputs, keyboard-navigable tabs.

11. Quality Gates

- ESLint + Prettier both apps; JSDoc on utils/services.
- Unit tests: `calculations.js` , `rollNumber.js` , line matchers/parser fixtures, auth service.

- Seed: 1 faculty + grade_scale. `README` + local Postgres setup + small logical commits per phase.

12. Work Plan + Definition of Done

Phases 1–8 as in project brief. Stop + summarize after each phase.
Done = faculty PDF upload populates all tables with no dupes; public lookup by enrollment + semester shows correct marks/SGPA/CGPA/ranks (no login); all topper views (overall, semester, college, subject) list names + CGPA/marks; same-PDF re-upload → clean 409.

13. Open Assumptions (to confirm in Phase 1)

See GitHub/issues or Phase-1 summary message – grade-scale source, semester derivation from SchemeID, single-file assumption, cookie + CORS in dev.